



Curvance

Security Assessment

December 9, 2025

Prepared for:

Chris

Curvance

Prepared by: **Simone Monica and Priyanka Bose**

Table of Contents

Table of Contents	1
Project Summary	3
Executive Summary	4
Project Goals	6
Project Targets	7
Project Coverage	8
Codebase Maturity Evaluation	11
Summary of Findings	13
Detailed Findings	15
1. GaugeManager's main pool-related functionalities can be temporarily made to revert	15
2. Protocol fee calculation rounds down	18
3. Harvest functionality could lock rewards if the Gelato bot executes an incorrect swap	20
4. Protocol fee can interfere with the slippage check, resulting in a denial of service	22
5. Missing validation of Velodrome pool type	24
6. Debt repayment calculation does not consider fee-on-transfer tokens	26
7. Contracts do not implement the corresponding interface	28
8. Redstone oracle unique signers threshold is low	30
9. Amount of LP tokens received when adding liquidity to Velodrome is not checked	31
10. rescueToken can be used to transfer double entryptoken tokens	34
11. removeSigner uses an incorrect index with the authorisedSigners array	36
12. removeSigner could remove an incorrect signer	38
13. removeSigner does not update _isAuthorisedSigner for the last signer when moved	40
14. removeSigner could inadvertently cause a denial of service to the oracle when decrementSignerThreshold is false	42
15. Unsafe casts in RedstoneCoreAdaptor could prevent the contract from working	44
16. RedstoneCoreAdaptor's constructor does not check for duplicate signers	46
17. Lack of proper error handling	47
A. Vulnerability Categories	48

B. Code Maturity Categories	50
C. Non-Security-Related Findings	52
D. Fix Review Results	54
Detailed Fix Review Results	55
E. Fix Review Status Categories	58
About Trail of Bits	59
Notices and Remarks	60

Project Summary

Contact Information

The following project manager was associated with this project:

Sam Greenup, Project Manager
sam.greenup@trailofbits.com

The following engineering director was associated with this project:

Josselin Feist, Engineering Director, Blockchain
josselin.feist@trailofbits.com

The following consultants were associated with this project:

Simone Monica, Consultant
simone.monica@trailofbits.com

Priyanka Bose, Consultant
priyanka.bose@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
January 9, 2025	Pre-project kickoff call
February 3, 2025	Status update meeting #1
February 11, 2025	Status update meeting #2
February 18, 2025	Delivery of report draft
February 18, 2025	Report readout meeting
May 21, 2025	Completion of fix review
May 23, 2025	Delivery of final comprehensive report
December 9, 2025	Review of additional fixes; delivery of updated report

Executive Summary

Engagement Overview

Curvance engaged Trail of Bits to review the security of the Curvance protocol, which offers liquidity provision, yield farming, position management through leveraging and deleveraging operations, and a reward mechanism for staked users.

A team of two consultants conducted the review from January 21 to February 18, 2025, for a total of eight engineer-weeks of effort. Our testing efforts focused on identifying issues that could disrupt the protocol's operations, cause financial losses for users, or lead to inconsistencies in leveraging and deleveraging positions. With full access to source code and documentation, we performed static and dynamic testing of the code, using automated and manual processes.

Observations and Impact

During this engagement, we found a high-severity issue regarding a missing check on the received LP amount when liquidity is added to a Velodrome pool, which happens when a user executes a leverage operation or when a bot harvests the rewards from the VelodromePToken contract (TOB-CRV-9). Other significant issues may cause a denial of service to the GaugeManager actions (TOB-CRV-1) or to the leverage and deleverage actions in the PositionManagementBase contract (TOB-CRV-4). We also found that the harvest functionality is missing validation of the swap's token output, which could cause rewards to be locked in the system (TOB-CRV-3). Finally, various issues in the RedstoneCoreAdaptor contract highlight its lack of testing (TOB-CRV-11, TOB-CRV-12, TOB-CRV-13, TOB-CRV-14, TOB-CRV-15, TOB-CRV-16).

Recommendations

Based on the codebase maturity evaluation and findings identified during the security review, Trail of Bits recommends that Curvance take the following steps:

- **Remediate the findings disclosed in this report.** These findings should be addressed as part of a direct remediation or any refactor that may occur when addressing other recommendations.
- **Improve the testing suite.** As highlighted through this review, there is at least one contract that is severely lacking testing; we recommend achieving complete line coverage of all contracts at the minimum. Mutation testing tools such as `slither-mutate` and `Necessist` can help improve the quality of the tests.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>
High	1
Medium	5
Low	5
Informational	6
Undetermined	0

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Configuration	1
Data Validation	12
Denial of Service	1
Undefined Behavior	3

Project Goals

The engagement was scoped to provide a security assessment of the Curvance contracts. Specifically, we sought to answer the following non-exhaustive list of questions:

- Do mathematical operations in core functions (deposit, withdrawal, position management) maintain precision and handle rounding correctly?
- Are there safeguards to ensure pool type validation (stable/volatile) cannot be bypassed during position management?
- Does the system implement sufficient slippage checks to protect users during token swaps and position adjustments?
- Are debt calculations precise and protected from manipulation during leverage/deleverage operations?
- Can the system prevent MEV attacks (like sandwich attacks) from affecting user position adjustments?
- Does the system implement proper error handling with descriptive error messages instead of generic reverts?
- Are there protections against potential denial-of-service vectors that could block critical system functions?

Project Targets

The engagement involved reviewing and testing the following target.

Curvance

Repository	https://github.com/curvance/curvance-contracts/
Version	53601bf579160d52f7c0987b216436596b610268
Type	Solidity
Platform	Ethereum

Project Coverage

This section provides an overview of the analysis coverage of the review, as determined by our high-level engagement goals. Our approaches included the following:

- **GaugeManager:** The GaugeManager contract is one of the core components of the Curvance protocol. It handles reward distribution by tracking balances from market positions (eTokens/pTokens) and manages both protocol-native CVE emissions and partner gauge rewards. It allows users to claim rewards directly, lock them in veCVE positions, or delegate them to another address to claim them on the users' behalf. We thoroughly investigated the reward calculation and distribution mechanics to ensure no rewards can be manipulated or incorrectly distributed across the system, and we investigated whether the pool functionality can be temporarily suspended. Additionally, we verified that all virtual balance tracking and state updates are executed accurately during deposits, withdrawals, and reward claims across different epochs and reward tokens.
- **VelodromeStableLPAdaptor and VelodromeVolatileLPAdaptor:** These contracts are oracle adaptors that add pricing support for Velodrome's stable and volatile LP tokens, respectively. These adaptors validate LP token types, manage asset support, and provide price feed information to the OracleManager contract. We thoroughly audited the contracts, focusing on proper validation of pool types when assets are added, state management when assets are added and removed, and access control mechanisms to ensure only authorized addresses can modify supported assets. Additionally, we verified that all oracle-related state changes are properly tracked when assets are added to or removed from the adaptors.
- **VelodromeStablePToken, VelodromeVolatilePToken, and PendleLPPToken:** These contracts manage LP positions for the Velodrome and Pendle protocols within Curvance. During our review, we looked for issues related to incorrect pool type identification (stable/volatile), reward collection, and exchange rate calculations. Additionally, we examined the validation processes for both input and output tokens during swap operations to ensure they prevent incorrect input/output tokens within the protocol, which could lead to token loss.
- **Position-management contracts:** These contracts enable users to manage leveraged positions through various operations including leveraging, deleveraging, and collateral swaps via integrations with the Velodrome and Pendle protocols. We investigated whether positions can be manipulated through exchange rate calculations, pool type mismatches (stable/volatile), or fee-on-transfer tokens. Additionally, we verified the accuracy of the debt calculations and collateral valuations during all position operations, focusing on rounding, slippage protection

mechanisms, proper oracle price usage, and precise token calculations during leveraging and deleveraging operations.

- **UniversalBalance:** The UniversalBalance contract serves as a core component of the Curvance protocol, providing a balance management system where users can either hold their tokens in a non-yielding state (sitting balance) or invest them to earn yield through the protocol's lending system (lending balance). We thoroughly investigated the deposit and withdrawal flows to ensure no tokens can be lost through rounding errors, locked due to state inconsistencies, or manipulated through exchange rate calculations. We also verified the accuracy of balance accounting between deposit and withdrawal flows, paying particular attention to slippage protection and precise token calculations during lending operations.
- **Multicall-checker contracts:** The PythAdaptorMulticallChecker and RedstoneAdaptorMulticallChecker contracts validate multicall operations, ensuring only approved function calls can be executed on authorized target contracts within the Curvance protocol. During our review, we verified the calldata validation mechanisms to ensure that only authorized function calls can be executed and that target contracts are properly verified.
- **Swap-checker contracts:** The PendleZapperCalldataChecker and VelodromeZapperCalldataChecker contracts provide a validation layer for zap operations within the Curvance protocol, ensuring that swap parameters and function calls match expected values when interacting with the Pendle and Velodrome protocols. We reviewed the validation mechanisms to ensure that the swap parameters, all token amounts, addresses, and function calls match the expected values during zap operations.
- **RedstoneCoreAdaptor:** This contract is built around the Redstone oracle with some custom modifications such as the ability for a permissioned user to add or remove valid signers. We reviewed the contract for issues related to the custom code and to ensure it follows best practices recommended by the Redstone documentation; among many issues with this contract, we found that it does not check for duplicate signers (TOB-CRV-16) and that it could allow an insufficient number of signers, which would cause a denial of service (TOB-CRV-14).

Coverage Limitations

Because of the time-boxed nature of testing work, it is common to encounter coverage limitations. The following list outlines the coverage limitations of the engagement and indicates system elements that may warrant further review:

- The Curvance team removed the GaugeManager contract from the scope of the engagement after the first week of review.

- The RedstoneCoreAdaptor was reviewed toward the end of the engagement. Given the number of issues we found, we recommend having it reviewed again more deeply, particularly the process of adding and removing assets.
- Some of the contracts in scope interact with other contracts that were out of scope, such as the EToken contract. The assumption is that those contracts work as intended.

Codebase Maturity Evaluation

Trail of Bits uses a traffic-light protocol to provide each client with a clear understanding of the areas in which its codebase is mature, immature, or underdeveloped. Deficiencies identified here often stem from root causes within the software development life cycle that should be addressed through standardization measures (e.g., the use of common libraries, functions, or frameworks) or training and awareness programs.

Category	Summary	Result
Arithmetic	The arithmetic in the codebase is mostly well documented, and where it is used inside an unchecked block, an explanation on why it is safe is given. For calculations where rounding could be problematic, the rounding direction is in favor of the protocol, and rounding directions are used consistently in the codebase.	Satisfactory
Auditing	State-changing functions emit events where appropriate, making it easy to identify when state changes are executed, though we did find that some error messages could be more user-friendly. Curvance is working on an incident response and monitoring plan, which was not reviewed during this engagement.	Satisfactory
Authentication / Access Controls	Access controls are applied correctly for the contracts in scope; in particular, some functions are callable only by the DAO or by a permissioned user. Certain actions, such as depositing for another recipient, have checks that the recipient delegated the sender. Additionally, in the position-management contracts, for interactions with a PToken or EToken contract when a user executes a leverage/deleverage action, there are callbacks between those contracts; also, these interactions are guarded such that only the corresponding contract can call the function.	Satisfactory
Complexity Management	The codebase is complex, as it has numerous features and some code is duplicated in favor of optimizations. This code duplication likely caused TOB-CRV-9 . In a few cases the same variable is reused with different semantics throughout the function, making it difficult to	Moderate

	follow and easier to make mistakes. As described in TOB-CRV-7 , there are issues with interface implementations, highlighting the need for a better approach to the project's development to avoid possible future issues.	
Decentralization	Once deployed, the system is immutable. There are permissioned users that are able to set system configuration; however, users can always withdraw their funds.	Satisfactory
Documentation	The protocol's website provides high-level documentation, and each function has its own NatSpec documentation. Overall, this documentation is helpful; however, in some cases, the NatSpec documentation is incorrect (see the related findings in appendix C).	Satisfactory
Low-Level Manipulation	Curvance uses assembly especially for gas-optimized reverts, for emitting events, and for getting function signatures/parameters. Additionally, it uses the Solady library, which extensively uses assembly; however, this library is out of scope for this review. All assembly code is documented.	Moderate
Testing and Verification	The codebase contains many unit and integration tests; however, not every possible state of the protocol is tested (for example, when the <code>protocolLeverageFee</code> is not <code>0</code>). Additionally, the <code>RedstoneCoreAdaptor</code> contract is lacking any testing on certain features, as highlighted by the number of findings around the addition or removal of signers. We recommend implementing complete line coverage at the minimum and then using mutation testing tools such as slither-mutate and Necessist to strengthen the tests.	Moderate
Transaction Ordering	There are interactions with third-party protocols to make swaps or add/remove liquidity from pools. These actions create MEV risks; to avoid potential big losses, accurate slippage protections and/or a check on a minimum amount received should be applied carefully. During the review, we found that in interactions with Velodrome, there are no such precautions (TOB-CRV-9).	Moderate

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	GaugeManager's main pool-related functionalities can be temporarily made to revert	Denial of Service	Medium
2	Protocol fee calculation rounds down	Undefined Behavior	Low
3	Harvest functionality could lock rewards if the Gelato bot executes an incorrect swap	Data Validation	Medium
4	Protocol fee can interfere with the slippage check, resulting in a denial of service	Data Validation	Medium
5	Missing validation of Velodrome pool type	Data Validation	Informational
6	Debt repayment calculation does not consider fee-on-transfer tokens	Data Validation	Informational
7	Contracts do not implement the corresponding interface	Configuration	Informational
8	Redstone oracle unique signers threshold is low	Data Validation	Informational
9	Amount of LP tokens received when adding liquidity to Velodrome is not checked	Data Validation	High
10	rescueToken can be used to transfer double entryptoken tokens	Undefined Behavior	Informational
11	removeSigner uses an incorrect index with the authorisedSigners array	Data Validation	Low
12	removeSigner could remove an incorrect signer	Data Validation	Low

13	removeSigner does not update _isAuthorisedSigner for the last signer when moved	Undefined Behavior	Medium
14	removeSigner could inadvertently cause a denial of service to the oracle when decrementSignerThreshold is false	Data Validation	Medium
15	Unsafe casts in RedstoneCoreAdaptor could prevent the contract from working	Data Validation	Low
16	RedstoneCoreAdaptor's constructor does not check for duplicate signers	Data Validation	Low
17	Lack of proper error handling	Data Validation	Informational

Detailed Findings

1. GaugeManager's main pool-related functionalities can be temporarily made to revert

Severity: Medium

Difficulty: High

Type: Denial of Service

Finding ID: TOB-CRV-1

Target: contracts/architecture/GaugeManager.sol

Description

The GaugeManager's functionalities, such as the deposit and withdrawal functions, will revert when a reward token is removed from the system but a pool still has rewards with that token.

The updatePool function calls the rewardAllocation function for each rewardToken associated with a pool token to update how many rewards were accumulated (figure 1.1). The rewardAllocation function (figure 1.2) will check if the reward token is approved and compute the rewards; if the reward token is not approved, the function reverts. The DAO approves or removes rewards by calling the addExtraRewardToken or removeExtraRewardToken function. Then, anyone can call the addExtraRewards function to associate a reward token with a pool token.

The issue is that when a reward token is removed with the removeExtraRewardToken function, the rewards for that token that are already associated with a pool are not removed, so the updatePool function will revert. This makes every function calling updatePool, such as deposit, withdraw, setEmissionRates and addExtraRewards, also revert.

```
function updatePool(address token) public {
    {
        ...
        address[] memory rewardTokensForMToken = rewardTokens[token];
        uint256 numTokens = rewardTokensForMToken.length;
        for (uint256 i; i < numTokens; ) {
            uint256 lastRewardTimestamp = _lastRewardTimestamp;

            // Query rewardToken then increment i.
            address rewardToken = rewardTokensForMToken[i++];
            uint256 index = rewardTokenToIndex[token][rewardToken];
```

```

uint256 accRewardPerShare = poolAccRewardPerShare[token][index];
uint256 lastEpoch = epochOfTimestamp(lastRewardTimestamp);
uint256 cachedCurrentEpoch = currentEpoch();
uint256 reward;

// Step through epochs and apply rewards.
while (lastEpoch < cachedCurrentEpoch) {
    uint256 endTimestamp = epochEndTime(lastEpoch);

    // Update rewards from lastRewardTimestamp to endTimestamp.
    reward =
        (RAY *
         (endTimestamp - lastRewardTimestamp) *
         rewardAllocation(token, lastEpoch, rewardToken)) /
        epochDuration;
    ...
}

// Update rewards from lastRewardTimestamp to current timestamp.
reward =
    (RAY *
     (block.timestamp - lastRewardTimestamp) *
     rewardAllocation(token, lastEpoch, rewardToken)) /
    epochDuration;
accRewardPerShare = accRewardPerShare + reward / totalDeposited;

poolAccRewardPerShare[token][index] = accRewardPerShare;
}

// Update pool storage.
poolLastRewardTimestamp[token] = block.timestamp;
}

```

*Figure 1.1: Snippet of the updatePool function
(contracts/architecture/GaugeManager.sol#L838-L912)*

```

function rewardAllocation(
    address token,
    uint256 epoch,
    address rewardToken
) public view returns (uint256) {
    if (rewardToken == cve) {
        return _epochInfo[epoch].tokenWeight[token];
    }

    uint256 index = rewardTokenToIndex[token][rewardToken];

```

```

    if (index == 0 || !approvedRewardTokens[rewardToken]) {
        revert GaugeManager__InvalidRewardToken();
    }

    return (epochDuration * _epochRewardPerSec[token][epoch][index]);
}

```

*Figure 1.2: The rewardAllocation function
([contracts/architecture/GaugeManager.sol#L754-L769](#))*

If this situation occurs, the DAO can call `addExtraRewardToken` to add the token back; however, this issue essentially means the token removal functionality is fundamentally broken and cannot be used.

Exploit Scenario

The DAO decides to remove tokenA by calling `removeExtraRewardToken`. Eve front runs that transaction by calling the `addExtraRewards` function to add tokenA as a reward token for a pool. Users cannot deposit nor withdraw from the pool since those functions now revert.

Recommendations

Short term, update the token removal functionality so that the removed token's `rewardToken` storage variable is removed for each pool. Note that the rewards allocated would still be present in the contract, so consider allowing the DAO to withdraw them.

Long term, redesign the `GaugeManager` contract to make it simpler. The Curvance team indicated plans to follow this recommendation.

2. Protocol fee calculation rounds down

Severity: Low

Difficulty: Low

Type: Undefined Behavior

Finding ID: TOB-CRV-2

Target:
contracts/market/position-management/PositionManagementBase.sol

Description

The leverage protocol fee calculation rounds down, favoring the users, instead of rounding up in favor of the protocol. This happens in the `onBorrow` function (figure 2.1) and in the `onRedeem` function.

```
function onBorrow(
    address borrowToken,
    address borrower,
    uint256 borrowAmount,
    LeverageStruct memory leverageData
) external override {
    ...

    // Take protocol fee, if any.
    uint256 fee = (borrowAmount * getProtocolLeverageFee()) / WAD;
    if (fee > 0) {
        leverageData.borrowAmount -= fee;
        SafeTransferLib.safeTransfer(
            borrowUnderlying,
            centralRegistry.daoAddress(),
            fee
        );
    }
    ...
}
```

Figure 2.1: Snippet of the `onBorrow` function

([contracts/market/position-management/PositionManagementBase.sol#L365-L374](#))

Exploit Scenario

Alice executes a leverage operation. The result of the fee calculation rounds down, making Alice pay a slightly lower fee than due.

Recommendations

Short term, use the `FixedPointMathLib` library to round up the fee calculation.

Long term, always have calculated amounts round in favor of the protocol. For example, the user's collateral should round down, the user's debt should round up, and any kind of fees the users have to pay should round up.

3. Harvest functionality could lock rewards if the Gelato bot executes an incorrect swap

Severity: Medium

Difficulty: High

Type: Data Validation

Finding ID: TOB-CRV-3

Target: contracts/market/token/VelodromeStablePToken.sol,
contracts/market/token/VelodromeVolatilePToken.sol,
contracts/market/token/PendleLPPToken.sol

Description

The VelodromeStablePToken and VelodromeVolatilePToken contracts have a harvest function, which harvests and compounds rewards. This functionality can be called only by a Gelato Network bot.

If the rewards harvested are not the underlying token, then a swap is needed; the swap inputToken and outputToken values are taken from the data argument sent by the bot. The inputToken is checked to ensure it is approved; however, there is no validation of the outputToken, which should be sd.token0 and is used to continue the process.

```
function harvest(
    bytes calldata data
) external override returns (uint256 yield) {
    ...
    // Swap from VELO to underlying tokens, if necessary.
    if (!rewardTokenIsUnderlying) {
        SwapperLib.Swap memory swapData = abi.decode(
            data,
            (SwapperLib.Swap)
        );
        if (!isApprovedAsset[swapData.inputToken]) {
            // this will be the same check: `swapData.inputToken != rewardToken`
            revert CompoundingPToken__UnapprovedAssetSwap();
        }
        SwapperLib.swapSafe(centralRegistry, swapData);
    }
}

uint256 totalAmountA = IERC20(sd.token0).balanceOf(address(this));
// Make sure swap was routed into token0, or that token0 is VELO.
if (totalAmountA == 0) {
    revert VelodromeStablePToken__SlippageError();
}
```

Figure 3.1: Snippet of the harvest function
([contracts/market/token/VelodromeStablePToken.sol#L181-L203](#))

If, for some reason, the Gelato bot uses an incorrect output token, then anyone can send a minimal amount of `sd.token0` to the contract and the check that occurs after the swap (figure 3.1) will pass. This means the rewards will be locked in the contract in whatever token the Gelato bot used to execute the trade.

Note that a similar issue exists in the harvest function of the `PendleLPPToken` contract.

Exploit Scenario

The Gelato bot calls the `harvest` function with a wrong `outputToken` for the swap. Eve notices the mistake and sends 5 tokens of `sd.token0` to the contract. The rewards are harvested and swapped to the wrong token, but the process continues as if only 5 tokens were the rewards. The rewards are locked in the contract.

Recommendations

Short term, have the `VelodromeStablePToken` and `VelodromeVolatilePToken` contracts validate that `swapData.outputToken` is equal to `sd.token0` when a swap is needed to harvest rewards. Have the `PendleLPPToken` contract validate that the output tokens are in the `isUnderlyingToken` variable.

Long term, for swaps that should result in a specific output token, always validate the output token on-chain; when a trading route is given as an argument, the protocol should not assume it is correctly set by an off-chain system. Additionally, if multiple swaps are needed, intermediate swaps should also be validated for their correspondence of the output token with the next swap's input token.

4. Protocol fee can interfere with the slippage check, resulting in a denial of service

Severity: Medium

Difficulty: High

Type: Data Validation

Finding ID: TOB-CRV-4

Target:
contracts/market/position-management/PositionManagementBase.sol

Description

In the PositionManagementBase contract, when a protocol fee (up to 2%) is charged during leveraging and deleveraging but the slippage tolerance is lower than the fee, transactions will always revert.

The depositAndLeverage function in the PositionManagementBase contract allows users to specify a slippage limit when setting up their leveraged position. When a user leverages their position, a protocol leverage fee is charged at the end of the borrowing period, as shown in figure 4.1.

```
// Take protocol fee, if any.
uint256 fee = (borrowAmount * getProtocolLeverageFee()) / WAD;
if (fee > 0) {
    leverageData.borrowAmount -= fee;
    SafeTransferLib.safeTransfer(
        borrowUnderlying,
        centralRegistry.daoAddress(),
        fee
    );
}
```

Figure 4.1: Snippet of the onBorrow function, showing the protocol fee deduction (contracts/market/position-management/PositionManagementBase.sol#L365-L374)

However, as shown in figure 4.2, this protocol fee can be as high as 2%. Therefore, if the slippage limit is below 2%, the slippage check will fail, causing the transaction to revert.

```
function setProtocolLeverageFee(uint256 value) external {
    // Leverage fee cannot be more than 2%.
    if (value > 200) {
        _revert(_PARAMETERS_MISCONFIGURED_SELECTOR);
    }
    // Convert the parameters from basis points to `WAD` format
```

```

// while inefficient we want to minimize potential human error
// as much as possible, even if it costs a bit extra gas on config.
protocolLeverageFee = _bpToWad(value);
...
}

```

Figure 4.2: Snippet of the `setProtocolLeverageFee` function, for setting the protocol leverage fee ([contracts/architecture/CentralRegistry.sol#L592–L605](#))

The same issue occurs when the slippage limit is set via the `setSlippage` function in the `CentralRegistry` contract. If the slippage limit is set to a value less than 2%, any subsequent leveraging or deleveraging transactions will fail because the protocol leverage fee exceeds the allowed slippage limit.

```

// Convert the parameters from basis points to `WAD` format
// while inefficient we want to minimize potential human error
// as much as possible, even if it costs a bit extra gas on config.
slippageLimit = _bpToWad(value);

```

Figure 4.3: Snippet of the `setSlippageLimit` function, for setting the slippage limit ([contracts/architecture/CentralRegistry.sol#L614–L622](#))

Exploit Scenario

The protocol admin sets a protocol fee percentage to be 2% and the slippage limit to be 1%. Alice sends a transaction to leverage her position. The transaction fails because the incurred slippage is more than 1% due to protocol fee deduction.

Recommendations

Short term, set a minimum allowed value for the slippage percentage that is greater than or equal to max allowable protocol fee percentage.

Long term, conduct comprehensive testing to verify that the protocol behaves correctly across all possible configurations and scenarios.

5. Missing validation of Velodrome pool type

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-CRV-5

Target:

contracts/market/position-management/PositionManagementVelodromeStable.sol,
contracts/market/position-management/PositionManagementVelodromeVolatile.sol

Description

The Velodrome pool type (stable/volatile) is not validated during leveraging and deleveraging operations. This allows users to deposit LP tokens from the wrong pool type, potentially leading to incorrect position management.

As shown in figure 5.1, in the `PositionManagementVelodromeStable` contract, the `_swapBorrowUnderlyingToCollateral` function fails to validate whether the Velodrome pool is the stable type. Without this check, a user could accidentally use tokens from the wrong pool type—for example, a user could use volatile pool tokens when the position requires stable pool tokens, or vice versa. This mismatch could lead to incorrect collateral valuations and potentially destabilize the position management system.

```
function _swapBorrowUnderlyingToCollateral(
    LeverageStruct memory leverageData,
    address recipient
) internal virtual override {
    address pool = leverageData.positionToken.underlying();
    address token0 = IVeloPair(pool).token0();
    address token1 = IVeloPair(pool).token1();

    address borrowUnderlying = leverageData.borrowToken.underlying();

    // If the token being borrowed isn't token0 we will need to swap
    // into it.
    if (borrowUnderlying != token0) {
        ...
        SwapperLib.swapSafe(centralRegistry, swapData);
    }
    ...
}
```

*Figure 5.1: Snippet of the `_swapBorrowUnderlyingToCollateral` function
(`contracts/market/position-management/PositionManagementVelodromeStable.sol#L60-L91`)*

The same issue exists in the `_swapBorrowUnderlyingToCollateral` function of the `PositionManagementVelodromeVolatile` contract.

Recommendations

Short term, add a check for the pool type (whether it is volatile or stable) in both of the affected position-management contracts.

Long term, conduct comprehensive testing across all scenarios and configurations to ensure the system provides expected outputs.

6. Debt repayment calculation does not consider fee-on-transfer tokens

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-CRV-6

Target:
contracts/market/position-management/PositionManagementBase.sol

Description

The debt repayment calculation does not account for tokens that deduct transfer fees (fee-on-transfer tokens). When such tokens are used for repayment, the actual amount received will be less than the calculated amount, resulting in incorrect debt settlement.

As shown in figure 6.1, the onRedeem function in the PositionManagementBase contract is responsible for repaying outstanding debt during collateral redemption.

```
// Repay debt.  
borrowToken.repayFor(redeemer, repayAmount);
```

Figure 6.1: Snippet of the onRedeem function in the PositionManagementBase contract (contracts/market/position-management/PositionManagementBase.sol#L510-L511)

However, as illustrated in figure 6.2, the implementation does not mention handling of fee-on-transfer tokens where the received amount differs from the sent amount during debt repayment transfers. This oversight means that if such tokens are used, the actual amount received will be less than the expected debt repayment, leading to incorrect debt settlement in the system.

```
SafeTransferLib.safeTransferFrom(  
    underlying,  
    payer,  
    address(this),  
    amount  
);  
// We calculate the new account and total borrow balances,  
// we check that amount is <= accountDebt so we can skip  
// underflow check here.  
unchecked {  
    _debtOf[account].principal = accountDebt - amount;  
}
```

Figure 6.2: Snippet of the _repay function in the EToken contract (contracts/market/token/EToken.sol#L1431-L1443)

Recommendations

Short term, update the documentation to explicitly state that fee-on-transfer tokens are not supported as borrow tokens. Alternatively, if fee-on-transfer tokens are intended to be supported, implement proper balance-tracking logic to account for transfer fees during token transfers.

Long term, carefully evaluate tokens before supporting them, and consider allowing only non-fee-on-transfer tokens. If fee-on-transfer tokens need to be supported, perform extensive testing to verify correct handling of token transfers and debt calculations across all possible scenarios and token configurations.

7. Contracts do not implement the corresponding interface

Severity: Informational	Difficulty: High
Type: Configuration	Finding ID: TOB-CRV-7
Target: *	

Description

Some contracts in the codebase implement an interface's functions but do not inherit from it. More importantly, there are places where a contract's address is cast to an interface but the contract does not implement that interface. If a function is added to the interface but not implemented in the contract, no compilation error would appear because the contract does not inherit the interface.

For example, the `BaseOracleAdaptor` contract does not inherit the `IOracleAdaptor` interface, but it implements all the functions.

```
abstract contract BaseOracleAdaptor {
```

*Figure 7.1: Declaration of the BaseOracleAdaptor contract
([contracts/market/token/VelodromeStablePToken.sol#L181-L203](#))*

```
interface IOracleAdaptor {
    /// @notice Called by OracleManager to price an asset.
    /// @param asset The address of the asset for which the price is needed.
    /// @param inUSD A boolean to determine if the price should be returned in
    ///             USD or not.
    /// @param getLower A boolean to determine if lower of two oracle prices
    ///                 should be retrieved.
    /// @return A structure containing the price, error status,
    ///         and the quote format of the price.
    function getPrice(
        address asset,
        bool inUSD,
        bool getLower
    ) external view returns (PriceReturnData memory);

    /// @notice Whether an asset is supported by the Oracle Adaptor or not.
    /// @dev Asset => Supported by adaptor.
    function isSupportedAsset(address asset) external view returns (bool);

    /// @notice Returns the adaptor's type.
    /// @dev Used by frontends to determine how to properly interact
    ///      with a supported asset.
    function adaptorType() external view returns (uint256);
```

```
}
```

*Figure 7.2: IOracleAdaptor interface
(contracts/interfaces/IOracleAdaptor.sol#L16–L39)*

The IOracleAdaptor interface is used multiple times as a casting destination in the OracleManager contract (figure 7.3), but OracleManager does not implement that interface.

```
function getAdaptorTypes(
    address asset
) external view returns (uint256, uint256) {
    ...

    return (IOracleAdaptor(adaptor).adaptorType(), 0);

    ...
    adaptorTypeA = isApprovedAdaptor[adaptor]
        ? IOracleAdaptor(adaptor).adaptorType()
        : 0;

    adaptor = assetPriceFeeds[asset][1];
    adaptorTypeB = isApprovedAdaptor[adaptor]
        ? IOracleAdaptor(adaptor).adaptorType()
        : 0;

    ...
}

function _addFeed(address asset, address feed) internal {
    ...
    // Validate that the feed supports the proposed asset.
    if (!IOracleAdaptor(feed).isSupportedAsset(asset)) {
        _revert(_INVALID_PARAMETER_SELECTOR);
    }
}
```

Figure 7.3: contracts/oracles/OracleManager.sol

Another example is that the OracleManager contract does not inherit the IOracleManager interface.

Recommendations

Short term, review all the contracts and make them inherit the correct interface if any.

8. Redstone oracle unique signers threshold is low

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-CRV-8

Target: contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol

Description

Redstone relies on multiple signers to ensure its data accuracy; the minimum number of signers can be overridden. The RedstoneCoreAdaptor contract sets the minimum number of signers (MINIMUM_SIGNER_THRESHOLD_ALLOWED) to 2; however, the Redstone documentation recommends setting it to at least 3.

```
/// @notice The smallest value that Redstone Core unique signer threshold  
///         can be inside Curvance.  
uint256 public constant MINIMUM_SIGNER_THRESHOLD_ALLOWED = 2;
```

*Figure 8.1: The minimum amount of signers for the Redstone oracle
(contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol#L41-L43)*

- Approximately 3 required unique signers is our recommended balance to be secure and minimize gas costs.

Figure 8.2: A snippet of the Redstone documentation

Recommendations

Short term, set MINIMUM_SIGNER_THRESHOLD_ALLOWED to 3.

Long term, when integrating with third-party protocols, read their documentation carefully, especially about integrations or recommendations.

9. Amount of LP tokens received when adding liquidity to Velodrome is not checked

Severity: High

Difficulty: Low

Type: Data Validation

Finding ID: TOB-CRV-9

Target: contracts/market/token/VelodromeStablePToken.sol,
contracts/market/token/VelodromeVolatilePToken.sol,
contracts/market/position-management/PositionManagementVelodromeStable.sol,
contracts/market/position-management/PositionManagementVelodromeVolatile.sol

Description

Curvance adds liquidity to a Velodrome pool when a user enters a leverage position with an LP token as collateral and when the bot harvests rewards in a VelodromePToken contract. However, in both cases, there is no check of the minimum number of LP tokens received by adding liquidity. (Note that for the details of this issue, the difference between a stable pool and a volatile pool is not important.)

The harvest function harvests the rewards and then swaps them into the token0 address. It then gets the pool on-chain reserves to calculate the optimal deposit amount of token0 and token1, swaps the necessary amount of token0 to token1, and finally adds liquidity. However, the function does not check the amount of the LP tokens received (the yield variable), so it is possible to sandwich calls to the harvest function to manipulate the pool on-chain reserves and make it deposit at an arbitrary valuation.

```
function harvest(  
    bytes calldata data  
) external override returns (uint256 yield) {  
    ...  
    // Pull reserve data so we can swap half of token0 into token1  
    // optimally.  
    (uint256 r0, uint256 r1, ) = IVeloPair(_asset).getReserves();  
    (uint256 reserveA, uint256 reserveB) = sd.token0 ==  
        IVeloPair(_asset).token0()  
        ? (r0, r1)  
        : (r1, r0);  
    // Feed library pair factory, lpToken, and stable = true,  
    // plus calculated data.  
    uint256 swapAmount = VelodromeLib._optimalDeposit(  
        address(sd.pairFactory),  
        _asset,
```

```

        totalAmountA,
        reserveA,
        reserveB,
        sd.decimalsA,
        sd.decimalsB,
        true
    );
    // Feed calculated data, and stable = true.
    VelodromeLib._swapExactTokensForTokens(
        address(sd.router),
        _asset,
        sd.token0,
        sd.token1,
        swapAmount,
        true
    );
    totalAmountA -= swapAmount;

    // Add liquidity to Velodrome lp with stable params.
    yield = VelodromeLib._addLiquidity(
        address(sd.router),
        sd.token0,
        sd.token1,
        true,
        totalAmountA,
        IERC20(sd.token1).balanceOf(address(this)), // totalAmountB
        VelodromeLib.VELODROME_ADD_LIQUIDITY_SLIPPAGE
    );
    ...

```

Figure 9.1: Snippet of the harvest function
([contracts/market/token/VelodromeStablePToken.sol#L207-L246](#))

The same process is executed when a user enters a leverage position using the `_swapBorrowUnderlyingToCollateral` function of the `PositionManagementVelodromeStable` and `PositionManagementVelodromeVolatile` contracts.

Exploit Scenario

The bot calls the harvest function. Eve sandwiches the transaction and profits at the cost of the `VelodromeStablePToken` holders.

Recommendations

Short term, have the bot send in the data argument of the harvest function a minimum amount of LP tokens that should be received, and have the harvest function check that amount against what is actually received. For the leverage function in the `PositionManagementVelodromeStable` and `PositionManagementVelodromeVolatile` contracts, enable the user to specify the minimum amount of LP tokens, and have the function use it to check the actual received amounts.

Long term, when implementing code that executes swaps, adds liquidity, or generally uses on-chain data that can be manipulated to compute an amount of tokens received, always consider the possibility of sandwich attacks and ensure the code validates the received amount against an expected value computed off-chain.

References

- [The Reverse LP Sandwich Arbitrage](#)

10. rescueToken can be used to transfer double entryptoken tokens

Severity: Informational

Difficulty: High

Type: Undefined Behavior

Finding ID: TOB-CRV-10

Target: contracts/architecture/UniversalBalance.sol

Description

The rescueToken function allows the DAO to transfer tokens that were sent incorrectly to the daoOperator address. It checks that the token to be transferred is not the underlying or the linkedEToken. However, this check does not work correctly for tokens that have double entryptokens because the other address could be specified to still transfer them.

```
function rescueToken(address token, uint256 amount) external {
    _checkDaoPermissions();
    address daoOperator = centralRegistry.daoAddress();

    if (token == address(0)) {
        if (amount == 0) {
            amount = address(this).balance;
        }

        SafeTransferLib.safeTransferETH(daoOperator, amount);
    } else {
        if (token == underlying || token == address(linkedEToken)) {
            _revert(_INVALID_PARAMETER_SELECTOR);
        }

        if (amount == 0) {
            amount = IERC20(token).balanceOf(address(this));
        }

        SafeTransferLib.safeTransfer(token, daoOperator, amount);
    }
}
```

*Figure 10.1: The rescueToken function
([contracts/architecture/UniversalBalance.sol#L393-L414](#))*

Exploit Scenario

A double entryptoken token that can be interacted with through address A, and address B is used as the underlying token through address A. The DAO is compromised and calls the rescueToken function for the underlying token with address B, transferring all the tokens to the daoOperator address.

Recommendations

Short term, if double entryptoken tokens will be supported, have the associated code get the balance of the underlying and linkedEToken before doing token transfers and check that those balances did not change after the transfers have executed. Note that this change should be applied to all the rescueToken functions present in the codebase. If double entryptoken tokens will not be supported, clearly document all the variations of ERC-20 tokens that are supported in the documentation.

11. removeSigner uses an incorrect index with the authorisedSigners array

Severity: Low

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-CRV-11

Target: contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol

Description

The removeSigner function of the RedstoneCoreAdaptor contract can remove an incorrect signer from the authorisedSigners array or revert when trying to remove the last signer in the array.

When removeSigner tries to remove an element whose index is different from the last signer's index, it takes the branch to move the last signer to the index position of the signer to be removed. However, the value in _isAuthorisedSigner corresponds to the signer position plus 1, so authorisedSigners[index] overwrites the signer after the signer to be removed.

```
function removeSigner(
    address currentSigner,
    bool decrementSignerThreshold
) external {
    ...
    uint256 index = _isAuthorisedSigner[currentSigner];
    ...
    uint256 lastSignerIndex = authorisedSigners.length - 1;

    // Switch array locations on authorised signer so we can pop
    // `currentSigner` from the end.
    if (index != lastSignerIndex) {
        authorisedSigners[index] = authorisedSigners[lastSignerIndex];
    }

    // Remove `currentSigner` from authorised signer list.
    authorisedSigners.pop();
    ...
}
```

Figure 11.1: Snippet of the removeSigner function

(contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol#L305-L344)

```
function addSigner(
    address newSigner,
    bool incrementSignerThreshold
) external {
```

```

...
uint256 index = _isAuthorisedSigner[newSigner];

/// Validate that `newSigner` is not already authorised.
if (index != 0) {
    revert RedstoneCoreAdaptor__InvalidConfiguration();
}

uint256 signerIndex = authorisedSigners.length;

// Add `newSigner` to quick access address mapping.
_isAuthorisedSigner[newSigner] = signerIndex + 1;
...

```

Figure 11.2: Snippet of the addSigner function

([contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol#L270-L295](#))

This finding is of low severity because the `authorisedSigners` array is never used for any semantic checks inside the protocol; it is used only for off-chain systems and users to track signers.

Exploit Scenario

The system has five signers. The first signer is removed through a call to the `removeSigner` function; however, the current last signer overwrites the second signer in the `authorisedSigners` array instead of the first one, which should have been removed.

Recommendations

Short term, correct the assignment of `authorisedSigners[index]` to `authorisedSigners[index-1]`. Note that depending on how other issues are fixed, the assigned value `authorisedSigners[lastSignerIndex]` may have to become `authorisedSigners[lastSignerIndex-1]`.

Long term, improve the testing suite, as it is lacking for the `RedstoneCoreAdaptor` contract.

12. removeSigner could remove an incorrect signer

Severity: Low

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-CRV-12

Target: contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol

Description

The removeSigner function of the RedstoneCoreAdaptor contract can remove an incorrect signer from the authorisedSigners array because it incorrectly computes the lastSignerIndex value.

The removeSigner function computes the lastSignerIndex value as the length of the authorisedSigners array minus 1 (figure 12.1). However, the index 0 is a special value that is not used; the indexes start from 1, as can be seen in the addSigner function (figure 12.2), where the assigned index is the length of the authorisedSigners array plus 1 (note that the new element has not been pushed to the array yet).

```
function removeSigner(
    address currentSigner,
    bool decrementSignerThreshold
) external {
    ...
    uint256 index = _isAuthorisedSigner[currentSigner];
    ...
    uint256 lastSignerIndex = authorisedSigners.length - 1;

    // Switch array locations on authorised signer so we can pop
    // `currentSigner` from the end.
    if (index != lastSignerIndex) {
        authorisedSigners[index] = authorisedSigners[lastSignerIndex];
    }

    // Remove `currentSigner` from authorised signer list.
    authorisedSigners.pop();
    ...
}
```

Figure 12.1: Snippet of the removeSigner function

(contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol#L305-L344)

```
function addSigner(
    address newSigner,
    bool incrementSignerThreshold
) external {
    ...
}
```

```
uint256 index = _isAuthorisedSigner[newSigner];

/// Validate that `newSigner` is not already authorised.
if (index != 0) {
    revert RedstoneCoreAdaptor__InvalidConfiguration();
}

uint256 signerIndex = authorisedSigners.length;

// Add `newSigner` to quick access address mapping.
_isAuthorisedSigner[newSigner] = signerIndex + 1;
...
```

Figure 12.2: Snippet of the addSigner function

(contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol#L270–L295)

This finding is of low severity because the `authorisedSigners` array is never used for any semantic checks inside the protocol; it is used only for off-chain systems and users to track signers.

Exploit Scenario

The system has five signers. A permissioned user calls `removeSigner` to remove the signer in the fourth position of the array. Its index is 4 and the `lastSignerIndex` is computed as 4, so the branch is not entered; `removeSigner` removes the last element from the array even though it was not the signer that should have been removed.

Recommendations

Short term, correct the `lastSignerIndex` to be `authorisedSigners.length`.

Long term, improve the testing suite, as it is lacking for the `RedstoneCoreAdaptor` contract.

13. removeSigner does not update _isAuthorisedSigner for the last signer when moved

Severity: Medium

Difficulty: Medium

Type: Undefined Behavior

Finding ID: TOB-CRV-13

Target: contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol

Description

The removeSigner function of the RedstoneCoreAdaptor contract does not update the _isAuthorisedSigner value of the last element when it is moved to the index of the signer to be removed from the authorisedSigners array. As a result, a signer added in the future will get the same _isAuthorisedSigner value, and when the signatures are counted to validate data sent to the oracle, their signatures will have the same index, so they can be counted only once.

```
function removeSigner(
    address currentSigner,
    bool decrementSignerThreshold
) external {
    ...
    uint256 index = _isAuthorisedSigner[currentSigner];
    ...
    uint256 lastSignerIndex = authorisedSigners.length - 1;

    // Switch array locations on authorised signer so we can pop
    // `currentSigner` from the end.
    if (index != lastSignerIndex) {
        authorisedSigners[index] = authorisedSigners[lastSignerIndex];
    }

    // Remove `currentSigner` from authorised signer list.
    authorisedSigners.pop();
    ...
}
```

Figure 13.1: Snippet of the removeSigner function

(contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol#L305-L344)

Exploit Scenario

A signer that is not the last in the array is removed through a call to the removeSigner function; the _isAuthorisedSigner value of the last element is not updated. Later, a signer is added through a call to the addSigner function and its _isAuthorisedSigner value is the same as the previous last element in the array.

Recommendations

Short term, have the `removeSigner` function update the `_isAuthorisedSigner` value of the last signer in the `authorisedSigners` array when moving it to another position.

Long term, improve the testing suite, as it is lacking for the `RedstoneCoreAdaptor` contract.

14. removeSigner could inadvertently cause a denial of service to the oracle when decrementSignerThreshold is false

Severity: Medium

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-CRV-14

Target: contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol

Description

The removeSigner function of the RedstoneCoreAdaptor contract allows the caller to decide whether to decrement the signer threshold when the signer is removed from the authorized list, by setting decrementSignerThreshold to true or to false (figure 14.1). However, if the caller does not decrement the threshold, _uniqueSignersThreshold could become greater than the number of signers (i.e., authorisedSigners) in the system.

```
function removeSigner(
    address currentSigner,
    bool decrementSignerThreshold
) external {
    _checkElevatedPermissions();
    ...

    // Remove `currentSigner` from authorised signer list.
    authorisedSigners.pop();

    if (decrementSignerThreshold) {
        // Make sure that decreasing the signer threshold would not pushed
        // signer requirement below minimum allowed inside the Curvature
        // Protocol.
        if (_uniqueSignersThreshold == MINIMUM_SIGNER_THRESHOLD_ALLOWED) {
            revert RedstoneCoreAdaptor__InvalidConfiguration();
        }

        _uniqueSignersThreshold--;
    }
}
```

Figure 14.1: Snippet of the removeSigner function

([contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol#L305-L344](#))

Exploit Scenario

The system has five signers, and the _uniqueSignersThreshold is 5. A permissioned user calls removeSigner to remove one signer and sets decrementSignerThreshold to false. The new system state is four signers, and because the

_uniqueSignersThreshold is 5, the oracle's data cannot be confirmed due not having enough signers.

Recommendations

Short term, modify `removeSigner` so that when `decrementSignerThreshold` is set to `false`, it checks that `authorisedSigners.length` is greater than or equal to `_uniqueSignersThreshold`.

Long term, improve the testing suite, as it is lacking for the `RedstoneCoreAdaptor` contract.

15. Unsafe casts in RedstoneCoreAdaptor could prevent the contract from working

Severity: Low

Difficulty: High

Type: Data Validation

Finding ID: TOB-CRV-15

Target: contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol

Description

The RedstoneCoreAdaptor contract's `getUniqueSignersThreshold` and `getAuthorisedSignerIndex` functions perform unsafe casts that could prevent the contract from working.

Both functions cast a `uint256` value to a `uint8`. The `getUniqueSignersThreshold` function casts the `_uniqueSignersThreshold` variable, while the `getAuthorisedSignerIndex` function casts the signer index in the array. These values can be more than 255 since there are no checks in the `addSigner` function that enforce an upper bound of signers in the system.

```
function getUniqueSignersThreshold() public view override returns (uint8) {
    return uint8(_uniqueSignersThreshold);
}
```

Figure 15.1: The `getUniqueSignersThreshold` function
([contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol#L357-L359](#))

```
function getAuthorisedSignerIndex(
    address signerAddress
) public view virtual override returns (uint8) {
    uint256 index = _isAuthorisedSigner[signerAddress];

    /// Validate that `signerAddress` is authorised.
    if (index == 0) {
        revert SignerNotAuthorised(signerAddress);
    }

    // Return authorised signer index.
    return uint8(index);
}
```

Figure 15.2: The `getAuthorisedSignerIndex` function
([contracts/libraries/external/redstone/PrimaryProdDataServiceConsumerBase.sol#L31-L43](#))

Exploit Scenario

The system has 256 signers. The last signer's index returned by the `getAuthorisedSignerIndex` function is 0 because the function unsafely casts 256 to a `uint8` value.

Recommendations

Short term, add an upper bound limit of signers in the `addSigner` function.

Long term, improve the testing suite, as it is lacking for the `RedstoneCoreAdaptor` contract.

16. RedstoneCoreAdaptor's constructor does not check for duplicate signers

Severity: Low

Difficulty: High

Type: Data Validation

Finding ID: TOB-CRV-16

Target: contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol

Description

The `_storeAuthorisedSigners` function is called in the constructor to store an initial set of signers, but it does not handle duplicate signers; if duplicate signers are present, an invalid configuration could result. In the worst case scenario, duplicate signers would prevent the `_uniqueSignersThreshold` value from being reached; however, this scenario can be fixed by adding new signers and removing the duplicate.

```
function _storeAuthorisedSigners(
    address[] memory signers
) internal override {
    uint256 numSigners = signers.length;
    address signer;

    for (uint256 i; i < numSigners; ++i) {
        signer = signers[i];

        _isAuthorisedSigner[signer] = i + 1;
        authorisedSigners.push(signer);

        emit RedstoneCoreSignerAdded(signer);
    }
}
```

Figure 16.1: Snippet of the `removeSigner` function

([contracts/oracles/adaptors/redstone/RedstoneCoreAdaptor.sol#L305-L344](#))

Exploit Scenario

The system is deployed with five signers, and five signers are needed to validate the data. However, one of the signers is a duplicate, and data cannot be confirmed because the threshold has not been reached.

Recommendations

Short term, check for duplicates in the `_storeAuthorisedSigners` function.

Long term, improve the testing suite, as it is lacking for the `RedstoneCoreAdaptor` contract.

17. Lack of proper error handling

Severity: Informational

Difficulty: Medium

Type: Data Validation

Finding ID: TOB-CRV-17

Target:
contracts/market/position-management/PositionManagementBase.sol

Description

The protocol uses panic reverts in certain scenarios instead of providing meaningful error messages. This makes it difficult for users to understand and resolve issues when interacting with the protocol.

For example, in the onRedeem function, as shown in figure 17.1, the code attempts to subtract repayAmount from the contract's token balance without first validating if sufficient balance exists. This operation can trigger an underflow panic when the balance is insufficient, instead of providing a clear error message.

```
uint256 remaining = IERC20(borrowUnderlying).balanceOf(address(this)) -  
    repayAmount;
```

Figure 17.1: Snippet of the onRedeem function

(contracts/market/position-management/PositionManagementBase.sol#L500-L501)

Recommendations

Short term, implement a check to validate that the contract's balance is greater than or equal to repayAmount before subtraction. The check should revert with clear, descriptive error messages when conditions are not met.

Long term, implement proper error handling to provide meaningful error messages during transaction failure.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Maturity Categories

The following tables describe the code maturity categories and rating criteria used in this document.

Code Maturity Categories	
Category	Description
Arithmetic	The proper use of mathematical operations and semantics
Auditing	The use of event auditing and logging to support monitoring
Authentication / Access Controls	The use of robust access controls to handle identification and authorization and to ensure safe interactions with the system
Complexity Management	The presence of clear structures designed to manage system complexity, including the separation of system logic into clearly defined functions
Cryptography and Key Management	The safe use of cryptographic primitives and functions, along with the presence of robust mechanisms for key generation and distribution
Decentralization	The presence of a decentralized governance structure for mitigating insider threats and managing risks posed by contract upgrades
Documentation	The presence of comprehensive and readable codebase documentation
Low-Level Manipulation	The justified use of inline assembly and low-level calls
Testing and Verification	The presence of robust testing procedures (e.g., unit tests, integration tests, and verification methods) and sufficient test coverage
Transaction Ordering	The system's resistance to transaction-ordering attacks

Rating Criteria	
Rating	Description
Strong	No issues were found, and the system exceeded industry standards.
Satisfactory	Minor issues were found, but the system is compliant with best practices.
Moderate	Some issues that may affect system safety were found.
Weak	Many issues that affect system safety were found.
Missing	A required component is missing, significantly affecting system safety.
Not Applicable	The category does not apply to this review.
Not Considered	The category was not considered in this review.
Further Investigation Required	Further investigation is required to reach a meaningful conclusion.

C. Non-Security-Related Findings

The following findings are not associated with specific vulnerabilities. However, addressing them may enhance code readability and prevent the introduction of vulnerabilities in the future.

- In the `onBorrow` function, the `borrowToken` is cast to `IPToken`, but it is an `IEToken`.

```
function onBorrow(  
    address borrowToken,  
    address borrower,  
    uint256 borrowAmount,  
    LeverageStruct memory leverageData  
) external override {  
    ...  
    address borrowUnderlying = IPToken(borrowToken).underlying();
```

Figure C.1: Snippet of the `onBorrow` function

([contracts/market/position-management/PositionManagementBase.sol#L352](#))

- The `reserveB`, `decimalsA`, and `decimalsB` arguments of the `_optimalDeposit` function can be `0`, as the function does not use them when computing the optimal deposit for a volatile pair.

```
uint256 swapAmount = VelodromeLib._optimalDeposit(  
    pairFactory,  
    pool,  
    totalAmountA,  
    reserveA,  
    reserveB,  
    decimalsA,  
    decimalsB,  
    false  
);
```

Figure C.2: Use of the `_optimalDeposit` function

([contracts/market/position-management/PositionManagementVelodromeVolatile.sol#L111-L120](#))

- The following check performed by the `VelodromeStablePToken` constructor can be simplified to `rewardToken == strategyData.token0` since at that point it is not possible for `rewardToken` to be `strategyData.token1`.

```
rewardTokenIsUnderlying = (rewardToken == strategyData.token0 ||  
    rewardToken == strategyData.token1);
```

*Figure C.3: Snippet of the VelodromeStablePToken constructor
(contracts/market/token/VelodromeStablePToken.sol#L120–L121)*

- In several locations, such as those shown in figures C.4 and C.5, the 4 bytes passed as an argument to the `_revert` function do not correspond to the error in the comment. Review all uses of this pattern and consider using a well-named constant variable rather than a magic value.
 - `bytes4(keccak256(bytes("CompoundingPToken__WithdrawMoreThanMax")))` is `0xfb0451f2`.

```
// We use a modified version of maxWithdraw with newly vested assets.
if (assets > _convertToAssets(balanceOf(owner), ta)) {
    // revert with "CompoundingPToken__WithdrawMoreThanMax".
    _revert(0x05203273);
}
```

*Figure C.4: Snippet of the `_withdraw` function
(contracts/market/token/CompoundingPToken.sol#L357–L361)*

- `bytes4(keccak256(bytes("CompoundingPToken__RedeemMoreThanMax")))` is `0xd7eb44a1`.

```
// Check whether `shares` is above max allowed redemption.
if (shares > maxRedeem(owner)) {
    // revert with "CompoundingPToken__RedeemMoreThanMax".
    _revert(0xcc3c42c0);
}
```

*Figure C.5: Snippet of the `_redeem` function
(contracts/market/token/CompoundingPToken.sol#L437–L441)*

- The following NatSpec comments should be corrected.
 - `contracts/market/token/BasePToken.sol#L634–L635`: “The number of assets a user would receive for converting assets” should be, “The number of assets a user would receive for converting shares”.
 - `contracts/architecture/CentralRegistry.sol#L610`: The NatSpec comment says, “can only have a maximum value of 2%”; however, the code does not have a maximum value, and that is the expected behavior.
 - `contracts/architecture/GaugeManager.sol#L204`: The NatSpec comment says, “Only the messaging hub can call this”; however, the function can be called also by the voting hub.

D. Fix Review Results

When undertaking a fix review, Trail of Bits reviews the fixes implemented for issues identified in the original report. This work involves a review of specific areas of the source code and system configuration, not comprehensive analysis of the system.

From May 19 to May 21, 2025, and in a follow-up review on December 9, 2025, Trail of Bits reviewed the fixes and mitigations implemented by the Curvance team for the issues identified in this report. We reviewed each fix to determine its effectiveness in resolving the associated issue.

In summary, Curvance has resolved all 17 issues disclosed in this report. For additional information, please see the Detailed Fix Review Results below.

ID	Title	Status
1	GaugeManager's main pool-related functionalities can be temporarily made to revert	Resolved
2	Protocol fee calculation rounds down	Resolved
3	Harvest functionality could lock rewards if the Gelato bot executes an incorrect swap	Resolved
4	Protocol fee can interfere with the slippage check, resulting in a denial of service	Resolved
5	Missing validation of Velodrome pool type	Resolved
6	Debt repayment calculation does not consider fee-on-transfer tokens	Resolved
7	Contracts do not implement the corresponding interface	Resolved
8	Redstone oracle unique signers threshold is low	Resolved
9	Amount of LP tokens received when adding liquidity to Velodrome is not checked	Resolved
10	rescueToken can be used to transfer double entryptoints tokens	Resolved

11	removeSigner uses an incorrect index with the authorisedSigners array	Resolved
12	removeSigner could remove an incorrect signer	Resolved
13	removeSigner does not update _isAuthorisedSigner for the last signer when moved	Resolved
14	removeSigner could inadvertently cause a denial of service to the oracle when decrementSignerThreshold is false	Resolved
15	Unsafe casts in RedstoneCoreAdaptor could prevent the contract from working	Resolved
16	RedstoneCoreAdaptor's constructor does not check for duplicate signers	Resolved
17	Lack of proper error handling	Resolved

Detailed Fix Review Results

TOB-CRV-1: GaugeManager's main pool-related functionalities can be temporarily made to revert

Resolved in [PR #255](#). The reward tokens functionality has been removed from the GaugeManager contract.

TOB-CRV-2: Protocol fee calculation rounds down

Resolved in [PR #255](#). The protocol fee calculation has been updated to round up.

TOB-CRV-3: Harvest functionality could lock rewards if the Gelato bot executes an incorrect swap

Resolved in [PR #255](#). The outputToken is now checked to be the expected one, as recommended.

TOB-CRV-4: Protocol fee can interfere with the slippage check, resulting in a denial of service

Resolved in [PR #255](#). The setSlippageLimit function requires the slippage limit to be at least 4%.

TOB-CRV-5: Missing validation of Velodrome pool type

Resolved in [PR #261](#). The _swapBorrowUnderlyingToCollateral function of the PositionManagementVelodromeVolatile and

PositionManagementVelodromeStable contracts now checks for the correctness of the pool type.

TOB-CRV-6: Debt repayment calculation does not consider fee-on-transfer tokens

Resolved in [PR #261](#). Documentation has been added to the BasePToken and EToken contracts that states the underlying asset cannot be a fee-on-transfer token.

TOB-CRV-7: Contracts do not implement the corresponding interface

Resolved in [PR #261](#) and [PR #287](#). The BaseOracleAdaptor contract now inherits the IOracleAdaptor interface; the OracleManager contract inherits the IOracleManager interface.

TOB-CRV-8: Redstone oracle unique signers threshold is low

Resolved in [PR #261](#). The oracle signers threshold has been updated to 3.

TOB-CRV-9: Amount of LP tokens received when adding liquidity to Velodrome is not checked

Resolved in [PR #263](#), [PR #287](#), and [PR #290](#). The `_swapBorrowUnderlyingToCollateral` function of the PositionManagementVelodrome contract now gets a minimum amount of LP tokens expected to be received by the user and checks it against what is actually received. A similar validation has also been added to the harvest function.

TOB-CRV-10: rescueToken can be used to transfer double entryptoken tokens

Resolved in [PR #263](#). Documentation has been added to state that double entryptoken tokens are not supported.

TOB-CRV-11: removeSigner uses an incorrect index with the authorisedSigners array

Resolved in [PR #261](#). The removeSigner function has been updated to use the correct index (`lastSignerIndex - 1`) when indexing the authorisedSigners array.

TOB-CRV-12: removeSigner could remove an incorrect signer

Resolved in [PR #261](#). The removeSigner function has been updated so that the `lastSignerIndex` variable is equal to `authorisedSigners.length`.

TOB-CRV-13: removeSigner does not update `_isAuthorisedSigner` for the last signer when moved

Resolved in [PR #261](#). The removeSigner function now updates `_isAuthorisedSigner` for the last signer when it is moved to a different position.

TOB-CRV-14: removeSigner could inadvertently cause a denial of service to the oracle when decrementSignerThreshold is false

Resolved in [PR #261](#). When called with `decrementSignerThreshold` equal to false, the removeSigner function now checks that the number of authorized signers (`authorisedSigners`) is at least the same as the value of `_uniqueSignersThreshold`.

TOB-CRV-15: Unsafe casts in RedstoneCoreAdaptor could prevent the contract from working

Resolved in [PR #261](#). An upper limit of 255 for the number of signers has been added to the addSigner function.

TOB-CRV-16: RedstoneCoreAdaptor's constructor does not check for duplicate signers

Resolved in [PR #261](#). The RedstoneCoreAdaptor's constructor now checks for duplicate signers and reverts with an error if it detects them.

TOB-CRV-17: Lack of proper error handling

Resolved in [PR #258](#). The onRedeem function now validates that the contract's balance is greater than or equal to repayAmount and reverts with a custom error if not.

E. Fix Review Status Categories

The following table describes the statuses used to indicate whether an issue has been sufficiently addressed.

Fix Status	
Status	Description
Undetermined	The status of the issue was not determined during this engagement.
Unresolved	The issue persists and has not been resolved.
Partially Resolved	The issue persists but has been partially resolved.
Resolved	The issue has been sufficiently resolved.

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review projects, supporting client organizations in the technology, defense, and finance industries and government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, and Zoom.

Trail of Bits also operates a center of excellence with regard to blockchain security. Notable projects include audits of Algorand, Bitcoin SV, Chainlink, Compound, Ethereum 2.0, MakerDAO, Matic, Uniswap, Web3, and Zcash.

To keep up to date with our latest news and announcements, please follow [@trailofbits](#) on X and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2025 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to Curvance under the terms of the project statement of work and has been made public at Curvance's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.

Test Coverage Disclaimer

Trail of Bits performed all activities associated with this project in accordance with a statement of work and an agreed-upon project plan.

Security assessment projects are time-boxed and often rely on information provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test software controls and security properties. These techniques augment our manual security review work, but each has its limitations. For example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. A project's time and resource constraints also limit their use.